16届国赛

```c
//ds18b20.c文件修改
float rd_temperature()
{
    unsigned char low,high;
    int raw;

    init_ds18b20();
    Write_DS18B20(0xcc);
    Write_DS18B20(0x44);
    Delay_OneWire(200);

    init_ds18b20();
    Write_DS18B20(0xcc);
    Write_DS18B20(0xbe);
    low=Read_DS18B20();
    high=Read_DS18B20();
  raw = ((int)high << 8) | low;
  if(raw & 0x8000)
      raw -= 65536;
  return raw * 0.0625f;
}

//main.c
#include <STC15F2K60S2.H>
#include "stdio.h"
#include "string.h"
#include "led.h"
#include "init.h"
#include "key.h"
#include "seg.h"
#include "ds1302.h"
#include "onewire.h"
#include "iic.h"
#include "ultrasound.h"
#include "uart.h"

// ============ 频率相关 ============
idata unsigned int  freq;
idata unsigned int  temp_freq;
idata unsigned int  freq_time_1s;
pdata unsigned int  freq_compare[5]={0,0,0,0,0};
idata unsigned char freq_index;
idata unsigned int  freq_sub;
idata unsigned char num_error_count;
bit got_freq_flag;

// ============ 串口 ============
bit                 uart_rx_flag;
idata unsigned char uart_rx_tick;
pdata unsigned char uart_rx_buf[64]={0};
idata unsigned char uart_rx_index;
idata unsigned char uart_mute_tick;

// ============ 按键 ============
idata unsigned char key_val,key_old,key_down,key_up;
idata unsigned char key_slow_down;

// ============ 数码管 ============
idata unsigned char seg_pos;
pdata unsigned char seg_buf[8]={10,10,10,10,10,10,10,10};
idata unsigned char seg_slow_down;
idata unsigned char seg_disp_mode;      //0-数据 1-参数 2-标定
idata unsigned char data_disp_mode;     //0-时间 1-液位 2-体积 3-重量
idata unsigned int  para_disp_mode;     //0-7

// ============ RTC ============
pdata unsigned char ucrtc[3]={23,59,50};

// ============ LED ============
```

```
pdata unsigned char ucled[8]={0,0,0,0,0,0,0,0};
bit                 L7_flash_flag;
idata unsigned char L7_flash_tick;
bit                 L8_flash_flag;
idata unsigned char L8_flash_tick;
idata unsigned int  time_3s;

// ============ AD / 标定 ============
idata unsigned char ad_slow_down;
pdata unsigned char biaoding_weight[5]={0,5,10,15,20};   // 单位kg
idata unsigned char biaoding_index;
pdata unsigned char V_ct_10x[5]={0,10,20,40,50};         // 临时标定值 0~50 对应0~5V
pdata unsigned char V_val_10x[5]={0,10,20,40,50};        // 实际标定值
idata int           V_jiaozhun_para;                     // 电压校准值（mV）

// ============ 温度 ============
idata unsigned int  T_slow_down;
idata int           T_val;                  // 温度整数部分（℃）
bit                 speed_ready;            // 声速是否有效
idata unsigned int  speed_cm_per_s;         // 声速 cm/s

// ============ 测距 / 液位 / 体积 / 重量 ============
idata unsigned char dist_slow_down;
idata unsigned int  yewei_cm;                // 液位（厘米整数）
bit                 yewei_error_flag;
idata unsigned int  dist_cm;                 // 测距值（厘米整数）
idata unsigned char space_baifenbi;          // 剩余空间百分比 0~100
idata unsigned int  yeti_tiji_10x;           // 液体体积（0.1立方米 × 10，即0.01立方米为单位）
idata unsigned int  weight_10x;              // 重量（0.1吨 × 10，即0.01吨为单位）

// ============ 8 个参数（单位统一为厘米整数）============
idata unsigned int  H1_cm = 100;             // 液位上限（cm）
idata unsigned int  H2_cm = 10;              // 液位下限（cm）
idata unsigned int  F = 2000;                // 频率阈值（Hz）
idata unsigned char S = 2;                   // 容器类型 0-球体 1-圆柱 2-长方体
idata unsigned int  r_cm = 100;              // 半径（cm）
idata unsigned int  L_cm = 100;              // 长（cm）
idata unsigned int  W_cm = 100;              // 宽（cm）
idata unsigned int  H_cm = 100;              // 高（cm）

// ============ 异常标志 ============
bit num_error_flag;       // 502
bit qushi_error_flag;     // 503
bit change_error_flag;    // 504
bit error_enable_flag;    // 继电器汇总

// ============ 异常上报去重标志 ============
unsigned char last_yewei=0xFF;   // 0x00-500 0x01-501 0xFF-正常
bit           last_502=0;
bit           last_504=0;
unsigned char last_503=0xFF;     // 0x00-下降 0x01-上升 0xFF-无


// ====================== 按键处理 ======================
void key_proc()
{
    unsigned char i;
    bit valid;
    if(key_slow_down<10) return;
    key_slow_down=0;
    key_val=key_read();
    key_down=key_val&(key_val^key_old);
    key_up  =~key_val&(key_val^key_old);
    key_old =key_val;

    if(key_down==4)
    {
        switch(seg_disp_mode)
        {
            case 0:
                seg_disp_mode=1;
                para_disp_mode=0;
                break;
```

```
            case 1:
                seg_disp_mode=2;
                biaoding_index=0;
                for(i=0;i<5;i++) V_ct_10x[i]=V_val_10x[i];
                break;
            case 2:
                valid=1;
                for(i=0;i<4;i++)
                    if(V_ct_10x[i]>=V_ct_10x[i+1]) { valid=0; break; }
                if(valid)
                    for(i=0;i<5;i++) V_val_10x[i]=V_ct_10x[i];
                seg_disp_mode=0;
                data_disp_mode=0;
                break;
        }
    }
    if(key_down==5)
    {
        switch(seg_disp_mode)
        {
            case 0: data_disp_mode =(++data_disp_mode)%4; break;
            case 1: para_disp_mode =(++para_disp_mode)%8; break;
            case 2: biaoding_index =(++biaoding_index)%5; break;
        }
    }
    if(seg_disp_mode==2)
    {
        if(key_down==8 && V_ct_10x[biaoding_index]<99) V_ct_10x[biaoding_index]++;
        if(key_down==9 && V_ct_10x[biaoding_index]>0 ) V_ct_10x[biaoding_index]--;
    }
}

// ====================== 数码管 ======================
void seg_proc()
{
    if(seg_slow_down<80) return;
    seg_slow_down=0;
    read_rtc(ucrtc);
    switch(seg_disp_mode)
    {
        case 0:
            switch(data_disp_mode)
            {
                case 0:  // 时间
                    seg_buf[0]=ucrtc[0]/10%10;
                    seg_buf[1]=ucrtc[0]%10;
                    seg_buf[2]=11;
                    seg_buf[3]=ucrtc[1]/10%10;
                    seg_buf[4]=ucrtc[1]%10;
                    seg_buf[5]=11;
                    seg_buf[6]=ucrtc[2]/10%10;
                    seg_buf[7]=ucrtc[2]%10;
                    break;
                case 1:  // 温度和液位
                    {
                        unsigned int temp_disp;
                        seg_buf[0]=10;
                        if(T_val>=0)
                        {
                            seg_buf[1]=(T_val>=100)?T_val/100%10:10;
                            seg_buf[2]=(T_val>=10)?T_val/10%10:10;
                            seg_buf[3]=T_val%10;
                        }
                        else
                        {
                            seg_buf[1]=11;
                            temp_disp=-T_val;
                            seg_buf[2]=(temp_disp>=10)?temp_disp/10%10:10;
                            seg_buf[3]=temp_disp%10;
                        }
                        seg_buf[4]=11;
                        // 液位显示：yewei_cm 单位cm，显示为米和厘米
                        seg_buf[5]=(yewei_cm/100)%10 + ',';   // 米十位加点
```

```
                seg_buf[6]=(yewei_cm/10)%10;           // 米个位
                seg_buf[7]=yewei_cm%10;                // 厘米
            }
            break;
        case 2:  // 空间和体积
            seg_buf[0]=(space_baifenbi>=100)?space_baifenbi/100%10:10;
            seg_buf[1]=(space_baifenbi>=10)?space_baifenbi/10%10:10;
            seg_buf[2]=space_baifenbi%10;
            seg_buf[3]=11;
            seg_buf[4]=(yeti_tiji_10x>=1000)?yeti_tiji_10x/1000%10:10;
            seg_buf[5]=(yeti_tiji_10x>=100)?yeti_tiji_10x/100%10:10;
            seg_buf[6]=yeti_tiji_10x/10%10+',';
            seg_buf[7]=yeti_tiji_10x%10;
            break;
        case 3:  // 温度和重量
            {
                unsigned int temp_disp;
                if(T_val>=0)
                {
                    seg_buf[0]=(T_val>=100)?T_val/100%10:10;
                    seg_buf[1]=(T_val>=10)?T_val/10%10:10;
                    seg_buf[2]=T_val%10;
                }
                else
                {
                    seg_buf[0]=11;
                    temp_disp=-T_val;
                    seg_buf[1]=(temp_disp>=10)?temp_disp/10%10:10;
                    seg_buf[2]=temp_disp%10;
                }
                seg_buf[3]=11;
                seg_buf[4]=(weight_10x>=1000)?weight_10x/1000%10:10;
                seg_buf[5]=(weight_10x>=100)?weight_10x/100%10:10;
                seg_buf[6]=weight_10x/10%10+',';
                seg_buf[7]=weight_10x%10;
            }
            break;
        }
        break;
    case 1:  // 参数界面
        seg_buf[0]=12;
        seg_buf[1]=para_disp_mode+1;
        seg_buf[2]=11;
        switch(para_disp_mode)
        {
            case 0:  // H1
                seg_buf[3]=10;
                seg_buf[4]=(H1_cm>=10000)?H1_cm/10000%10:10;
                seg_buf[5]=(H1_cm>=1000)?H1_cm/1000%10:10;
                seg_buf[6]=H1_cm/100%10+',';
                seg_buf[7]=H1_cm/10%10;
                break;
            case 1:  // H2
                seg_buf[3]=10;
                seg_buf[4]=(H2_cm>=10000)?H2_cm/10000%10:10;
                seg_buf[5]=(H2_cm>=1000)?H2_cm/1000%10:10;
                seg_buf[6]=H2_cm/100%10+',';
                seg_buf[7]=H2_cm/10%10;
                break;
            case 2:  // F
                seg_buf[3]=(F>=10000)?F/10000%10:10;
                seg_buf[4]=(F>=1000)?F/1000%10:10;
                seg_buf[5]=(F>=100)?F/100%10:10;
                seg_buf[6]=(F>=10)?F/10%10:10;
                seg_buf[7]=F%10;
                break;
            case 3:  // S
                seg_buf[3]=10;seg_buf[4]=10;seg_buf[5]=10;seg_buf[6]=10;
                seg_buf[7]=S;
                break;
            case 4:  // r
                seg_buf[3]=10;
                seg_buf[4]=(r_cm>=10000)?r_cm/10000%10:10;
```

```
                seg_buf[5]=(r_cm>=1000)?r_cm/1000%10:10;
                seg_buf[6]=r_cm/100%10+',';
                seg_buf[7]=r_cm/10%10;
                break;
            case 5:  // L
                seg_buf[3]=10;
                seg_buf[4]=(L_cm>=10000)?L_cm/10000%10:10;
                seg_buf[5]=(L_cm>=1000)?L_cm/1000%10:10;
                seg_buf[6]=L_cm/100%10+',';
                seg_buf[7]=L_cm/10%10;
                break;
            case 6:  // W
                seg_buf[3]=10;
                seg_buf[4]=(W_cm>=10000)?W_cm/10000%10:10;
                seg_buf[5]=(W_cm>=1000)?W_cm/1000%10:10;
                seg_buf[6]=W_cm/100%10+',';
                seg_buf[7]=W_cm/10%10;
                break;
            case 7:  // H
                seg_buf[3]=10;
                seg_buf[4]=(H_cm>=10000)?H_cm/10000%10:10;
                seg_buf[5]=(H_cm>=1000)?H_cm/1000%10:10;
                seg_buf[6]=H_cm/100%10+',';
                seg_buf[7]=H_cm/10%10;
                break;
            }
            break;
        case 2:  // 标定界面
            seg_buf[0]=13;
            seg_buf[1]=biaoding_weight[biaoding_index]/10%10;
            seg_buf[2]=biaoding_weight[biaoding_index]%10;
            seg_buf[3]=seg_buf[4]=seg_buf[5]=10;
            seg_buf[6]=V_ct_10x[biaoding_index]/10%10+',';
            seg_buf[7]=V_ct_10x[biaoding_index]%10;
            break;
    }
}

// ====================== LED + 继电器 ======================
void led_proc()
{
    if(seg_disp_mode==0)
    {
        ucled[0]=(data_disp_mode==0);
        ucled[1]=(data_disp_mode==1);
        ucled[2]=(data_disp_mode==2);
        ucled[3]=(data_disp_mode==3);
    }
    else
    {
        ucled[0]=ucled[1]=ucled[2]=ucled[3]=0;
    }
    ucled[4]=yewei_error_flag;
    ucled[5]=num_error_flag;
    ucled[6]=L7_flash_flag;
    ucled[7]=L8_flash_flag;
    error_enable_flag = yewei_error_flag||num_error_flag||qushi_error_flag||change_error_flag;
    relay(error_enable_flag);
}

bit error_report_ready()
{
    return (uart_rx_index==0 && uart_rx_flag==0 && uart_mute_tick==0);
}

// ====================== 称重 ======================
void ad_proc()
{
    unsigned char i;
    unsigned int  V_mv;
    int           V_temp_mv;
    unsigned char voltage;
    if(ad_slow_down<100) return;
```

```
    ad_slow_down=0;
    // 按用户要求，保留两次读取的写法
    ad_read(0x03);
    voltage = ad_read(0x03);
    V_mv = (unsigned int)((unsigned long)voltage * 1000 / 51);
    V_temp_mv = (int)V_mv + V_jiaozhun_para;   // mV

    if(V_temp_mv <= V_val_10x[0]*100)      { weight_10x=0;   return; }
    if(V_temp_mv >= V_val_10x[4]*100)      { weight_10x=200; return; }

    for(i=0;i<4;i++)
    {
        if(V_temp_mv <= V_val_10x[i+1]*100)
        {
            weight_10x = (i*50) + (unsigned long)(V_temp_mv - V_val_10x[i]*100) * 50 / ((V_val_10x[i+1]-
V_val_10x[i])*100);
            break;
        }
    }
}

// ====================== 温度（声速 + 电压校准）======================
void T_proc()
{
        float temp;
        if(T_slow_down < 300) return;
        T_slow_down = 0;

        temp = rd_temperature();
        if(temp > 125.0f) temp -= 4096.0f;
        if(temp < -40.0f || temp > 85.0f)
                temp = (float)T_val;
        else
                T_val = (int)temp;

        /* 若按满分代码对齐，用 +255；若严格按题面公式，可改回 +225 */
        V_jiaozhun_para = (int)(0.04f * temp * temp - 10.0f * temp + 225.0f);

        /* 你的 ut_wave_data() 需要 m/s，不要乘 100 */
        speed_cm_per_s = (unsigned int)(330.0f + 0.6f * temp);
        speed_ready = 1;
}

// ====================== 测距 + 液位 + 体积 + 剩余空间 ======================
void dist_proc()
{
        float volume_cm3;
        unsigned int total_h;

        if(dist_slow_down < 100) return;
        dist_slow_down = 0;
        if(!speed_ready) return;

        dist_cm = ut_wave_data(speed_cm_per_s);
        read_rtc(ucrtc);

        switch(S)
        {
                case 0:  /* 球体 */
                        total_h = 2 * r_cm;

                        if(r_cm == 0) return;

                        if(dist_cm >= total_h)
                                yewei_cm = 0;
                        else
                                yewei_cm = total_h - dist_cm;

                        if(yewei_cm > total_h)
                                yewei_cm = total_h;

                        volume_cm3 = 3.14f * yewei_cm * yewei_cm * (3 * r_cm - yewei_cm) / 3.0f;
                        yeti_tiji_10x = (unsigned int)(volume_cm3 / 100000.0f + 0.5f);
```

```
434
435                     if(dist_cm >= total_h)
436                             space_baifenbi = 100;
437                     else
438                             space_baifenbi = (unsigned char)(
439                                     (float)dist_cm * dist_cm * (3 * r_cm - dist_cm) * 100.0f /
440                                     (4.0f * r_cm * r_cm * r_cm)
441                             );
442                     break;
443
444             case 1:  /* 圆柱体 */
445                     if(r_cm == 0 || H_cm == 0) return;
446
447                     if(dist_cm >= H_cm)
448                             yewei_cm = 0;
449                     else
450                             yewei_cm = H_cm - dist_cm;
451
452                     volume_cm3 = 3.14f * r_cm * r_cm * yewei_cm;
453                     yeti_tiji_10x = (unsigned int)(volume_cm3 / 100000.0f + 0.5f);
454
455                     space_baifenbi = (dist_cm >= H_cm) ? 100 :
456                             (unsigned char)((unsigned long)dist_cm * 100 / H_cm);
457                     break;
458
459             case 2:  /* 长方体 */
460                     if(L_cm == 0 || W_cm == 0 || H_cm == 0) return;
461
462                     if(dist_cm >= H_cm)
463                             yewei_cm = 0;
464                     else
465                             yewei_cm = H_cm - dist_cm;
466
467                     yeti_tiji_10x = (unsigned int)(((unsigned long)L_cm * W_cm * yewei_cm + 50000UL) / 100000UL);
468
469                     space_baifenbi = (dist_cm >= H_cm) ? 100 :
470                             (unsigned char)((unsigned long)dist_cm * 100 / H_cm);
471                     break;
472     }
473
474     if(yewei_cm > H1_cm)
475     {
476             yewei_error_flag = 1;
477             if(last_yewei != 0 && error_report_ready())
478             {
479                     last_yewei = 0;
480                     printf("[500,%02bu%02bu%02bu,%u.%02u]\r\n",
481                                 ucrtc[0], ucrtc[1], ucrtc[2],
482                                 yewei_cm / 100, yewei_cm % 100);
483             }
484     }
485     else if(yewei_cm < H2_cm)
486     {
487             yewei_error_flag = 1;
488             if(last_yewei != 1 && error_report_ready())
489             {
490                     last_yewei = 1;
491                     printf("[501,%02bu%02bu%02bu,%u.%02u]\r\n",
492                                 ucrtc[0], ucrtc[1], ucrtc[2],
493                                 yewei_cm / 100, yewei_cm % 100);
494             }
495     }
496     else
497     {
498             yewei_error_flag = 0;
499             last_yewei = 0xFF;
500     }
501 }
502
503 // ====================== 频率异常检测 ======================
504 void freq_proc()
505 {
506     if(got_freq_flag==0) return;
```

```
    got_freq_flag=0;
    read_rtc(ucrtc);     // 502 数值异常：越限事件立即上报一次；恢复正常后才允许再次上报
    if(freq > F)
    {
        num_error_flag=1;
        qushi_error_flag=0;
        last_503=0xFF;
        change_error_flag=0;
        last_504=0;
        L7_flash_tick=0;
        L7_flash_flag=0;
        L8_flash_tick=0;
        L8_flash_flag=0;
        time_3s=0;

        if(!last_502 && uart_rx_index==0 && uart_rx_flag==0)
        {
            last_502=1;
            printf("[502,%02bu%02bu%02bu,%u]\r\n", ucrtc[0], ucrtc[1], ucrtc[2], freq);
        }
    }
    else
    {
        num_error_flag=0;
        last_502=0;
        num_error_count=0;
    }

    // 503 趋势异常（滑动窗口5个点）
    if(freq_index >= 5)
    {
        if(freq_compare[0]<freq_compare[1] && freq_compare[1]<freq_compare[2] &&
           freq_compare[2]<freq_compare[3] && freq_compare[3]<freq_compare[4])
        {
            qushi_error_flag=1;
            if(last_503!=1 && error_report_ready() && !num_error_flag && !yewei_error_flag)
            {
                last_503=1;
                printf("[503,%02bu%02bu%02bu,1]\r\n", ucrtc[0], ucrtc[1], ucrtc[2]);
            }
        }
        else if(freq_compare[0]>freq_compare[1] && freq_compare[1]>freq_compare[2] &&
                freq_compare[2]>freq_compare[3] && freq_compare[3]>freq_compare[4])
        {
            qushi_error_flag=1;
            if(last_503!=0 && error_report_ready() && !num_error_flag && !yewei_error_flag)
            {
                last_503=0;
                printf("[503,%02bu%02bu%02bu,0]\r\n", ucrtc[0], ucrtc[1], ucrtc[2]);
            }
        }
        else
        {
            qushi_error_flag=0;
            last_503=0xFF;
        }
    }

    // 504 剧烈变化
    if(change_error_flag)
    {
        if(!last_504 && error_report_ready() && !num_error_flag && !yewei_error_flag)
        {
            last_504=1;
            printf("[504,%02bu%02bu%02bu,%u]\r\n", ucrtc[0], ucrtc[1], ucrtc[2], freq_sub);
        }
    }
    else
    {
        last_504=0;
    }
}
```

```
// ====================== 串口辅助函数 ======================
//比较两个字符串是否相同
//两个数相同返回1 不同返回0
bit str_eq(unsigned char *a, unsigned char *b)
{
    unsigned char i=0;
    while(a[i]!='\0' && b[i]!='\0')
    {
        if(a[i]!=b[i]) return 0;
        i++;
    }
    return (a[i]==b[i]);
}

// 解析"米"字符串为厘米整数，例如 "2.0" -> 200cm, "1.5" -> 150cm
unsigned int parse_m_to_cm(unsigned char *str)
{
    unsigned int integer=0, decimal=0;
    unsigned char i=0, dot=0xFF;//没有找到小数点 则dot置为0xff
        //1、先找小数点
    while(str[i]!='\0')
    {
        if(str[i]=='.') { dot=i; break; }
        i++;
    }
    i=0;
    if(dot!=0xFF)
    {
        while(i<dot)
        {
            integer = integer*10 + (str[i]-'0');
            i++;
        }
        i++;
        if(str[i]>='0' && str[i]<='9')
            decimal = str[i]-'0';
    }
    else //没有小数点那就当整数来对待
    {
        while(str[i]!='\0')
        {
            integer = integer*10 + (str[i]-'0');
            i++;
        }
    }
    return integer*100 + decimal*10;
}

//把串口接收到的数据转化为整数
//返回数据所对应的整数值
unsigned int parse_int(unsigned char *str)
{
    unsigned int r=0;
    unsigned char i=0;
    while(str[i]>='0' && str[i]<='9')
    {
        r = r*10 + (str[i]-'0');
        i++;
    }
    return r;
}

//按(标识,数据) (标识:数据)格式读取接收到的串口数据
//返回值:  ')'所对应的下标值
unsigned char extract_one(unsigned char start, unsigned char *id, unsigned char *val)
{
    unsigned char i=start, j=0;
    while(i<uart_rx_index && uart_rx_buf[i]!='(') i++;
    if(i>=uart_rx_index) return 0xFF;
    i++;
    j=0;
    while(i<uart_rx_index && uart_rx_buf[i]!=',' && uart_rx_buf[i]!=':')
        id[j++] = uart_rx_buf[i++];
```

```
    id[j]='\0';
    if(i>=uart_rx_index) return 0xFF;
    i++;
    j=0;
    while(i<uart_rx_index && uart_rx_buf[i]!=')')
        val[j++] = uart_rx_buf[i++];
    val[j]='\0';
    if(i>=uart_rx_index) return 0xFF;
    return i;
}

//将接收到的(标识,数据) 或（T：时间）或（标识,？）进行解析
void handle_param(unsigned char *id, unsigned char *val, unsigned char *ok, unsigned char *err)
{
    unsigned int t;

        if(str_eq(val, "?"))
        {
                if(str_eq(id, "H1"))       { printf("(H1,%u.%u)\r\n", H1_cm/100, H1_cm/10%10); *ok=1; }
                else if(str_eq(id, "H2")) { printf("(H2,%u.%u)\r\n", H2_cm/100, H2_cm/10%10); *ok=1; }
                else if(str_eq(id, "F"))   { printf("(F,%u)\r\n", F); *ok=1; }
                else if(str_eq(id, "S"))   { printf("(S,%bu)\r\n", S); *ok=1; }
                else if(str_eq(id, "r"))   { printf("(r,%u.%u)\r\n", r_cm/100, r_cm/10%10); *ok=1; }
                else if(str_eq(id, "L"))   { printf("(L,%u.%u)\r\n", L_cm/100, L_cm/10%10); *ok=1; }
                else if(str_eq(id, "W"))   { printf("(W,%u.%u)\r\n", W_cm/100, W_cm/10%10); *ok=1; }
                else if(str_eq(id, "H"))   { printf("(H,%u.%u)\r\n", H_cm/100, H_cm/10%10); *ok=1; }
                else *err=1;
                return;
        }

    if(str_eq(id,"T"))
    {
        unsigned char h,m,s,k;
        for(k=0;k<6;k++)
            if(val[k]<'0'||val[k]>'9') { *err=1; return; }
        if(val[6]!='\0') { *err=1; return; }
        h = (val[0]-'0')*10 + (val[1]-'0');
        m = (val[2]-'0')*10 + (val[3]-'0');
        s = (val[4]-'0')*10 + (val[5]-'0');
        if(h>23||m>59||s>59) { *err=1; return; }
        ucrtc[0]=h; ucrtc[1]=m; ucrtc[2]=s;
        set_rtc(ucrtc);
        *ok=1;
        return;
    }

    if(str_eq(id,"H1"))
    {
        t = parse_m_to_cm(val);
        if(t < H2_cm) { *err=1; return; }
        H1_cm = t; *ok=1; return;
    }
    if(str_eq(id,"H2"))
    {
        t = parse_m_to_cm(val);
        if(t > H1_cm) { *err=1; return; }
        H2_cm = t; *ok=1; return;
    }
    if(str_eq(id,"F")) { F = parse_int(val); last_502=0; num_error_count=0; freq_index=0; temp_freq=0;
qushi_error_flag=0; last_503=0xFF; change_error_flag=0; last_504=0; *ok=1; return; }
    if(str_eq(id,"S"))
    {
        t = parse_int(val);
        if(t>2) { *err=1; return; }
        S = (unsigned char)t; *ok=1; return;
    }
        if(str_eq(id,"r")) { t = parse_m_to_cm(val); if(t == 0) { *err = 1; return; } r_cm = t; *ok = 1; return; }
        if(str_eq(id,"L")) { t = parse_m_to_cm(val); if(t == 0) { *err = 1; return; } L_cm = t; *ok = 1; return; }
        if(str_eq(id,"W")) { t = parse_m_to_cm(val); if(t == 0) { *err = 1; return; } W_cm = t; *ok = 1; return; }
        if(str_eq(id,"H")) { t = parse_m_to_cm(val); if(t == 0) { *err = 1; return; } H_cm = t; *ok = 1; return; }
    *err=1;
}

```

```c
void uart_proc()
{
    unsigned char pos=0;
    unsigned char id[5];
    unsigned char val[10];
    unsigned char ok=0, err=0;

    if(uart_rx_index==0) return;
    if(uart_rx_tick >= 10)
    {
        uart_rx_flag=0;
        uart_rx_tick=0;
        while(pos < uart_rx_index)
        {
            pos = extract_one(pos, id, val);
            if(pos==0xFF) { err=1; break; }
            handle_param(id, val, &ok, &err);
            pos++;
        }
        if(err && ok)  printf("WARN\r\n");
        else if(err)   printf("ERROR\r\n");
        else if(ok)    printf("OK\r\n");
        uart_rx_index=0;
        uart_mute_tick=200;
    }
}

// ====================== 定时器 ======================
void Timer0_Init(void)
{
    AUXR &= 0x7F;
    TMOD &= 0xF0;
    TMOD |= 0x05;
    TL0 = 0x00;
    TH0 = 0x00;
    TF0 = 0;
    TR0 = 1;
}

void Timer1_Init(void)
{
    AUXR &= 0xBF;
    TMOD &= 0x0F;
    TL1 = 0x18;
    TH1 = 0xFC;
    TF1 = 0;
    TR1 = 1;
    ET1 = 1;
    EA = 1;
}

void Timer1_Isr(void) interrupt 3
{
    key_slow_down++;
    seg_slow_down++;
    ad_slow_down++;
    T_slow_down++;
    dist_slow_down++;

    if(seg_buf[seg_pos]>=20)
        seg_disp(seg_pos, seg_buf[seg_pos]-',', 1);
    else
        seg_disp(seg_pos, seg_buf[seg_pos], 0);
    seg_pos = (++seg_pos)%8;

    if(++freq_time_1s == 1000)
    {
        freq_time_1s = 0;
        freq = TH0<<8 | TL0;
        got_freq_flag = 1;
        // 504 变化量计算
        if(temp_freq != 0)
        {
```

```
            if(freq > temp_freq) freq_sub = freq - temp_freq;
            else                 freq_sub = temp_freq - freq;
            if(freq_sub > 1000)
            {
                change_error_flag = 1;
                time_3s = 0;
            }
        }
        temp_freq = freq;

        // 滑动窗口更新
        if(freq_index < 5)
            freq_compare[freq_index++] = freq;
        else
        {
            freq_compare[0]=freq_compare[1];
            freq_compare[1]=freq_compare[2];
            freq_compare[2]=freq_compare[3];
            freq_compare[3]=freq_compare[4];
            freq_compare[4]=freq;
        }
        TH0 = TL0 = 0;
    }

    // L7闪烁（503）
    if(qushi_error_flag)
    {
        if(++L7_flash_tick==100)
        {
            L7_flash_tick=0;
            L7_flash_flag = !L7_flash_flag;
        }
    }
    else
    {
        L7_flash_tick=0;
        L7_flash_flag=0;
    }

    // L8闪烁 + 3s锁定（504）
    if(change_error_flag)
    {
        if(++L8_flash_tick==200)
        {
            L8_flash_tick=0;
            L8_flash_flag = !L8_flash_flag;
        }
        if(++time_3s >= 3000)
        {
            time_3s=0;
            L8_flash_tick=0;
            L8_flash_flag=0;
            change_error_flag=0;
        }
    }
    else
    {
        time_3s=0;
        L8_flash_tick=0;
        L8_flash_flag=0;
    }

    if(uart_rx_flag) uart_rx_tick++;
    if(uart_mute_tick) uart_mute_tick--;
    led_disp(ucled);
}

void Uart1_Isr(void) interrupt 4
{
    if(RI)
    {
        uart_rx_flag = 1;
        uart_rx_tick = 0;
```

```
        uart_rx_buf[uart_rx_index++] = SBUF;
        RI = 0;
        if(uart_rx_index >= 64)
            uart_rx_index = 0;
    }
}

// ====================== 主函数 ======================
void main()
{
    system_init();
    set_rtc(ucrtc);
    Timer0_Init();
    Uart1_Init();
    Timer1_Init();
    while(1)
    {
        key_proc();
        freq_proc();
        uart_proc();
        seg_proc();
        ad_proc();
        T_proc();
        dist_proc();
        led_proc();
    }
}
```

uart_mute_tick 是我加的“串口静默计时器”，作用是：刚回复完串口命令后，短时间内先别主动上报普通异常，避免异常帧插到 OK / 查询返回附近。

两行分别是：

```
uart_mute_tick = 200;
```

放在 uart_proc() 处理完一条串口命令后。意思是设置一个 200ms 左右的静默窗口。因为你的定时器中断大约 1ms 进一次，所以 200 约等于 0.2 秒。

```
if(uart_mute_tick) uart_mute_tick--;
```

放在 Timer1_Isr() 里，每 1ms 递减一次。减到 0 后，说明静默窗口结束，普通异常上报又可以发送。

它配合这个函数使用：

```
bit error_report_ready()
{
    return (uart_rx_index==0 && uart_rx_flag==0 && uart_mute_tick==0);
}
```

也就是说，只有串口没有正在接收、且静默计时结束，500/501/503/504 这类主动上报才允许发。

注意：我后来让 502 不受 uart_mute_tick 限制，因为测试要求在指定频率异常时刻马上看到 [502,...]，如果也等静默窗口，容易漏测。